Lesson 9 - Quick Reference Card: Authentication & Authorization

🚀 Essential Commands

```bash
# Install Laravel Breeze
composer require laravel/breeze --dev
php artisan breeze:install blade
npm install && npm run dev
php artisan migrate

# Create Policy
php artisan make:policy PostPolicy
php artisan make:policy PostPolicy --model=Post

# Create Middleware
php artisan make:middleware EnsureUserIsAdmin

# Install Sanctum (API tokens)
composer require laravel/sanctum
php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"
php artisan migrate
```

🔐 Authentication Basics

```php
// Check if user is logged in
auth()->check()          // Returns true/false
auth()->guest()          // Returns true if not logged in
auth()->user()           // Get current user
auth()->id()             // Get current user ID

// Login
Auth::attempt(['email' => $email, 'password' => $password])
Auth::attempt($credentials, $remember = true)  // With remember me
Auth::login($user)
Auth::loginUsingId(1)

// Logout
Auth::logout()
$request->session()->invalidate()
```

```
$request->session()->regenerateToken()
```

🛡️ Authorization - Gates

```php
// Define Gate (in AppServiceProvider)
Gate::define('update-post', function (User $user, Post $post) {
    return $user->id === $post->user_id;
});

// Use Gate
Gate::allows('update-post', $post)      // Returns true/false
Gate::denies('update-post', $post)       // Returns true/false
Gate::authorize('update-post', $post)    // Throws 403 if denied

// In Blade
@can('update-post', $post)
    Edit
@endcan

@cannot('update-post', $post)


You cannot edit


@endcannot
```

📋 Policies

```php
// Create Policy
php artisan make:policy PostPolicy --model=Post

// Policy Methods
public function viewAny(User $user): bool
public function view(?User $user, Post $post): bool
public function create(User $user): bool
public function update(User $user, Post $post): bool
public function delete(User $user, Post $post): bool
public function restore(User $user, Post $post): bool
```

```
public function forceDelete(User $user, Post $post): bool

// Use Policy in Controller
$this->authorize('update', $post);
$this->authorize('create', Post::class);

// Use Policy in Blade
@can('update', $post)
    Edit
@endcan
```

🔒 Middleware

```php
// In routes
Route::middleware('auth')->group(function () {
    Route::get('/dashboard', [DashboardController::class, 'index']);
});

Route::middleware('guest')->group(function () {
    Route::get('/login', [LoginController::class, 'create']);
});

Route::middleware(['auth', 'verified'])->group(function () {
    Route::get('/premium', [PremiumController::class, 'index']);
});

// In Controller
public function __construct()
{
    $this->middleware('auth');
    $this->middleware('auth')->only(['create', 'store']);
    $this->middleware('auth')->except(['index', 'show']);
}
```

🔑 Password Management

```php
// Hash password
use Illuminate\Support\Facades\Hash;
```

```
$hashed = Hash::make('password');

// Check password
Hash::check('plain-text', $hashedPassword)

// Validation rule for current password
$request->validate([
    'current_password' => 'required|current_password',
    'password' => 'required|string|min:8|confirmed',
]);
```

🎫 API Tokens (Sanctum)

```php
// In User Model
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens;
}

// Create token
$token = $user->createToken('token-name')->plainTextToken;

// Create token with abilities
$token = $user->createToken('token-name', ['posts:create', 'posts:update'])
              ->plainTextToken;

// Protect routes
Route::middleware('auth:sanctum')->group(function () {
    Route::get('/user', function (Request $request) {
        return $request->user();
    });
});

// Check token ability
if ($request->user()->tokenCan('posts:create')) {
    // Can create
}

// Delete token (logout)
$request->user()->currentAccessToken()->delete();
```

👤 Blade Directives

```php
// Authentication
@auth


Welcome {{ auth()->user()->name }}


@endauth

@guest
    Login
@endguest

// Authorization
@can('update', $post)
    Edit
@endcan

@cannot('delete', $post)


You cannot delete


@endcannot

// Check role
@if(auth()->user()->role === 'admin')
    Admin Panel
@endif
```

🔚 Session Management

```php
// Regenerate session (prevent session fixation)
$request->session()->regenerate();

// Invalidate session
$request->session()->invalidate();
```

```
// Regenerate CSRF token
$request->session()->regenerateToken();

// Complete logout
Auth::logout();
$request->session()->invalidate();
$request->session()->regenerateToken();
```

📝 Common Patterns

Login Controller
```php
public function store(Request $request)
{
    $credentials = $request->validate([
        'email' => 'required|email',
        'password' => 'required',
    ]);

    if (Auth::attempt($credentials, $request->boolean('remember'))) {
        $request->session()->regenerate();
        return redirect()->intended('dashboard');
    }

    return back()->withErrors(['email' => 'Invalid credentials']);
}
```

Register Controller
```php
public function store(Request $request)
{
    $validated = $request->validate([
        'name' => 'required|string|max:255',
        'email' => 'required|email|unique:users',
        'password' => 'required|min:8|confirmed',
    ]);

    $user = User::create([
        'name' => $validated['name'],
        'email' => $validated['email'],
        'password' => Hash::make($validated['password']),
    ]);
```

```
    Auth::login($user);
    return redirect('/dashboard');
}
```

Resource Controller with Authorization
```php
public function update(Request $request, Post $post)
{
    $this->authorize('update', $post);

    $post->update($request->validated());

    return redirect()->route('posts.show', $post);
}
```

⚡ Quick Tips

1. **Always use `$this->authorize()` in controllers** for authorization checks
2. **Regenerate session** on login to prevent session fixation
3. **Use Hash::make()** for passwords, never plain text
4. **Check auth()->check()** before using auth()->user()
5. **Use Policies** for complex authorization logic
6. **Middleware 'auth'** protects routes requiring login
7. **Middleware 'guest'** for login/register pages only
8. **Remember to invalidate session** on logout

🎯 Common Use Cases

```php
// Check if user owns the post
return $user->id === $post->user_id;

// Check if admin
return $user->role === 'admin';

// Check if verified email
return $user->email_verified_at !== null;

// Check if can moderate
return in_array($user->role, ['admin', 'moderator']);
```

```
// Super admin can do everything
public function before(User $user, string $ability): ?bool
{
    if ($user->role === 'super-admin') {
        return true;
    }
    return null;
}
```

✅ Lesson 9 Checklist

- [ ] Understand Authentication vs Authorization
- [ ] Install and configure Laravel Breeze
- [ ] Implement login/register/logout
- [ ] Create and use Gates
- [ ] Create and use Policies
- [ ] Use auth middleware
- [ ] Implement password management
- [ ] Use Sanctum for API authentication
- [ ] Understand session management

💡 Key Takeaways

1. **Authentication** = Who are you? (Login/Register)
2. **Authorization** = What can you do? (Permissions)
3. **Gates** = Simple permission checks
4. **Policies** = Organized permissions for models
5. **Middleware** = Protect routes
6. **Sanctum** = API token authentication
7. Always **authorize()** in controllers
8. Always **regenerate session** on login

🔗 Quick Links

- [Main Lesson](./README.md)
- [English README](./README-EN.md)
- [Practice Guide](./PRACTICE-GUIDE-EN.md)
- [Full Exam](./FULL-EXAM-100-QUESTIONS.md)
- [Next Lesson](../lesson-10/README.md)